5) Delta Lake

I) Foundations of Delta Lake

A) The problem with Parquet file

B) Delta Architecture

C) How Delta ensures ACID

SECTION-1 : Foundations of Delta Lake

A) The real problem : Data Lakes with Parquet are Broken

⇒ Before Delta, companies used

Data Lake (S3 | ADLS | HDFS)
↓
Parquet files
↓
Spark jobs

problem 1 : No ACID Transactions :

- Parquet does not support Transactions
- So; no rollback
  No recovery

problem 2 : No concurrency control :

Imagine two engineers running jobs;
↳ Job A writing to /sales
↳ Job B also writing to /sales

parquet does not :
- Lock files
- Track transactions
- Detect conflicts

→ results :
- Data Inconsistency
- Duplicate data
- Lost updates

problem 3 : No updates or Deletes :

- Parquet is immutable.
- If we want to update / delete rows;
  we must :
  ↳ read entire dataset
  ↳ modify rows.
  ↳ rewrite entire dataset

Problem 4 - No schema Enforcement

↳ If we add changes to the schema; parquet will allow in that time

↳ But when we try the next time to access the data in the files; shows; schema mismatch error

{ • No enforcement
• No validation }

Problem 5 - No Time Travel :

- Someone deletes data accidentally.
- No built-in version history.

Problem 6 - Small File Problem :

↳ Every write creates many small files.

What Does Delta Lake Do?

↳ Delta lake adds a transaction layer on top of parquet.

Note

↳ { Delta does Not replace parquet,
Delta uses parquet internally }

Delta adds :

- ACID transactions
- Schema enforcement
- Schema evolution
- Time Table
- Optimized updates/deletes
- Concurrency Control
- Data skipping
- File Compaction

Delta = Parquet files + Transaction log + Snapshot engine

B) Delta Architecture (&) _delta_log :

Normal parquet Table

```
/sales
    part-0001.parquet
    part-0002.parquet
    part-0003.parquet
```

Delta Table:

```
/sales
    part-0001.parquet
    part-0002.parquet
    part-0003.parquet
    _delta_log/
        0000....0.json
        0000....1.json
        0000....2.json
```

What is _delta_log ?

↳ It is a transaction log.

It describes:

- Files added
- Files removed
- metadata changes
- Schema
- partition info

- DELTA never mutates old files directly
- It tracks changes via log.

Instead of:

```
df.write.parquet("/sales")
```

we run:

```
df.write.format("delta")\
    .mode("append")\
    .save("/sales")
```

Inside that JSON:

```
{
    "add" : {
        'path' : 'part-0004.parquet',
        'size' : 12345,
        'modificiationTime' : ____
    }
}
```

Versioning:

```
000.....0.json  ---> version 0
000.....1.json  ---> version 1
000.....2.json  ---> version 2
```

```
spark.read.format('delta').load('/sales')
```

↳ Delta:
- ↳ reads all JSON logs
- ↳ reconstructs current state
- ↳ Builds a snapshot

That snapshot = current table view

What is snapshot?

Logical view of table at a specific version

It is build by:
- ↳ Applying all JSON commits in order
- ↳ Ignoring removed files
- ↳ keeping active files.

What are checkpoints?

↳ over time, log files grow:

- 0000.json
- 0001.json
- ⋮
- 00500.json

(reading 500 logs every time is slow)

↳ so Delta creates:

000500.checkpoint.parquet

↳ checkpoint contains compacted snapshot state

↳ so Delta reads:

- latest checkpoint
- logs after checkpoint

Checkpoint:

(A compacted snapshot of the table state at a specific version)

Where is checkpoint stored?

```
/sales
    part-00001.parquet
    part-00002.parquet
    _delta_log/
        000.....0.json
        000.....1.json
        ⋮
        000.....10.checkpoint.parquet
        000.....11.json
        ⋮
```

When is checkpoint created?

: By default:

Every 10 commits.

What is Inside checkpoint?

- Checkpoint is NOT JSON
- It is a Parquet file
- It is a Materialized snapshot of table state at that version.

It contains:
- ↳ All active files
- ↳ Metadata
- ↳ Schema
- ↳ Partition info
- ↳ Remove actions applied

Why checkpoint is critical for streaming?

In streaming:
- ↳ Micro-batches create frequent commits
- ↳ version count grows fast.
- ↳ stream restarts often.

> If no checkpoint:

Every restart would:
- replay thousands of JSON logs
- slow startup
- Increase latency

Checkpoint ensures:
- fast recovery
- faster stream restart
- lower initialization overhead.

Snapshot:

- It is not a physical file
- It is In-Memory representation of table state at a specific version.

snapshot contains:
- List of active data files
- current schema
- current metadata
- partition information

Where can you see snapshot?

```
dt = DeltaTable.forPath(spark, "/sales")
dt.history().show()
```

We can manually inspect "_delta_log"

```
dbutils.fs.ls("/sales/_delta_log")
```

c) How Delta Ensures ACID:

ACID:

Atomicity	All or nothing commit
Consistency	Schema + constraints enforced
Isolation	Concurrent writes don't corrupt data
Durability	once committed, data persists

How are those ensured?

Atomicity:

- Commit happens only when, JSON log file successfully written.
- If log write fails:
  - ↳ parquet files exist
  - ↳ But no commit entry.

Isolation

- Each Job operates on: A consistent snapshot version.
  - ↳ Even if new commits happen during execution, job does not see partial updates

Consistency:

- ↳ (schema enforcement)

→ If someone tries to write diff. schema:

(Delta validates schema before commit.)

If mismatch:

write fails.

Durability:

once JSON log file written:

- ↳ commit is permanent
- ↳ stored in (S3/ADLS)
- ↳ survives restart.